Software Implementation Procedure

Vehicle Washing Management System (Vehicle Washing)

Planning, Environment Preparation, Deployment, Initial Data Load, Acceptance Testing, Training, Go-Live, and Rollback

Brayan Estiven Patiño Cabrera (Project Leader), Juan Camilo Guilombo Osorio (Frontend Leader), and Merari Urbano Quintero (Backend Leader)

Technologist in Software Analysis and Development (ADSO), Servicio Nacional de Aprendizaje (SENA), Training Center: Industria

Ficha 3145556

Instructor: Karol Daniela Correa

October 26, 2026

Document Type: Implementation Procedure (Development Team, IT Administrator, and Business Owner) · Version 2.9

Version Control

Table 1 records the change history of this document. Each time the implementation strategy, the target environment, or a step of the procedure changes, a row must be added with the version, the date, the description, and the person responsible.

Table 1

Version History of the Implementation Procedure

Version	Date	Description of the change	Responsible
1.0	2026-10-09	Creation of the procedure (in Spanish and English) from the code of the eleven repositories (*develop* branch as of October 9, 2026), *lavarapido-infra/docker-compose.yml*, and the project documentation on deployment, environments, operations, and quality.	Development team
2.0	2026-10-16	Review against the code of October 16, 2026, and the public server already set up. Updated: public server on Azure with Caddy, HTTPS, and the web served with nginx, APK distribution, first administrator through environment variables, 57 tables, operations-service queue, payment-service health, web and end-to-end tests in the entry criteria, automatic assignment in the smoke tests, business data in the initial load, and implementation risks.	Development team
2.1	2026-10-17	Review against the code of October 17, 2026. Stated that the mobile app needs a new APK because of the native modules of the inspection report (section 11.7, Phase 6).	Development team
2.2	2026-10-19	Continuous integration with GitHub Actions in the 10 repositories and deployment from GitHub: the scope (6.2) and risk R-I09 were updated.	Development team
2.3	2026-10-20	Review against the code of October 20, 2026: criterion E2 with the tests of October 19 (655 automated and 19 end-to-end) and RabbitMQ queues corrected (the operations-service queue has no dead-letter queue) and communication plan (the panel does not send general announcements).	Development team
2.4	2026-10-21	Review against the code and the server of October 21, 2026: criterion E2 with the tests of that day (689 automated and 19 end-to-end), automatic pre-implementation backup on the public server (phase 3, monitoring, and rollback), smoke test H10 (the token of a closed session stops working), new variables with defaults, app updates without a new APK, and risk R-I11.	Development team
2.5	2026-10-21	Review against the code of the evening of October 21, 2026: criterion E2 with the final run of that day (707 automated tests and 19 end-to-end), operations-service *FOLLOW_UP_** variables, smoke test H11 (the administrator's "To close" list), and updated configuration rule. Confirmed bookings with an operator that nobody registered are no longer closed as "No-show" on their own: the administrator resolves them.	Development team
2.6	2026-10-23	Review against *develop* of October 23, 2026 and version *v1.0.0*: cloning that works (payment-service from *develop* until its first release), rollback without the nonexistent *v0.9.0* tag and without *update-server.sh*, pre-update backup without *COMPRESSION* (SQL Server Express does not support it), criterion E2 with the tests of October 23 (825 automated and 13 of the mobile app; the 19 end-to-end tests run only locally), criterion E4 with the payment and mobile exception, Compose variables (ACO-14 closed), *INTERNAL_API_KEY* required for the operator, demo accounts through variables, warning about ports on the local network, and continuous deployment prepared and not tested (risk R-I04). Includes the content of Merari Urbano's *fix/A-05*, *fix/A-08*, *fix/B-02*, and *fix/E-20* branches.	Development team
2.7	2026-10-24	Review against *develop* of October 24, 2026, which runs on the public server since that day: criterion E2 with the tests of October 24 (1,119 automated and 73 in the mobile app, 1,192 in total), criterion E6 met (no P0 or P1 open), cloning with *BRANCH=develop* until *v1.1.0*, rollback with *rollback.sh* and the *:prev* images that *update-server.sh* keeps, restore with *payment-migrate*, the protection that prevents downgrading, new variables (*AUTH_RATE_LIMIT_**, *TRUSTED_PROXIES*, *ROUTE_*_TIMEOUT*, *START_LATE_LIMIT*, and *INTERNAL_API_KEY* and *BOOKING_SERVICE_URL* in customer-service), and continuous deployment merged into *develop*, with the secrets still missing (R-I04).	Development team
2.8	2026-10-26	Payment at booking, deployed to the public server on October 26, 2026: *INTERNAL_API_KEY* is now also required by payment-service, and new booking-service variables (*BOOKING_CASH_ACTIVE_LIMIT*, *BOOKING_PAYMENT_**, *PAYMENT_SERVICE_URL*) in Table 6.	Development team
2.9	2026-10-26	R-I04: continuous deployment is active since October 26, 2026 (AC-90).	Development team

Note. Dates are written in the ISO 8601 format (year-month-day). Prepared by the development team.

Table of Contents

Version Control 2
4. Introduction 4
4.1 Meaning of the Term "Implementation" 4
4.2 Sources of This Document 4
5. Objective 4
5.1 General Objective 4
5.2 Specific Objectives 4
6. Scope 4
6.1 What This Procedure Covers 4
6.2 What This Procedure Does Not Cover 5
7. Roles and Responsibilities 5
8. Implementation Strategy 6
8.1 Starting Point 6
8.2 Selected Strategy 6
8.3 Relationship With the ISO/IEC/IEEE 12207 Processes 6
9. Target Environment and Components 6
10. Entry Criteria 7
11. Step-by-Step Procedure 8
11.1 Phase 0: Planning and Approval 8
11.2 Phase 1: Environment Preparation 8
11.3 Phase 2: Building and Verifying the Release 11
11.4 Phase 3: Pre-Implementation Backup 11
11.5 Phase 4: Infrastructure and Database 12
11.6 Phase 5: Backend Services and Gateway 12
11.7 Phase 6: Client Applications 12
11.8 Phase 7: Initial Business Configuration 13
11.9 Phase 8: Smoke Tests 13
11.10 Phase 9: Acceptance Testing 14
11.11 Phase 10: Training 14
11.12 Phase 11: Go-Live and Post-Implementation Support 15
11.13 Phase 12: Closure 15
12. Rollback Plan 15
13. Release Configuration Management 16
14. Implementation Risks 17
15. Communication Plan 18
16. Exit Criteria 18
17. Glossary 19
References 20
18. Annexes 20
Annex A. Pre-Implementation Checklist 21
Annex B. Acceptance Test Cases 22
Annex C. Acceptance Record Template 23
Annex D. Implementation Log Template 24
Annex E. Command Summary 25

Note. Numbering starts at 4 because sections 1 to 3 are the cover, the version control, and the table of contents, which have no number in their title. It is kept this way so that the cross-references between documents by section number (for example, "Installation Manual, section 10") remain valid.

4. Introduction

The Vehicle Washing Management System (Vehicle Washing), internally called *LavaRapido*, replaces the manual procedure of a car wash (a paper schedule, work distributed at the manager's discretion, and payments without a record) with a digital platform for bookings, operator assignment, execution and rating of services, verification of payments by QR code, and a loyalty points program. Technically, it consists of a gateway, six microservices (five in Java 21 with Spring Boot 4 and one in C# with .NET 8), a SQL Server 2022 database, a RabbitMQ message broker, and two clients: an Angular 21 web application and an Android application in React Native with Expo (Vehicle Washing Development Team, 2026a).

This document defines the procedure to **implement** a release of the system in its target environment: how it is planned, what must be met before starting, in which order the components are deployed, how the initial business data is loaded, how the result is verified and accepted, how users are trained, how the system goes live, and how it is rolled back if something fails. It starts from the premise that an implementation must be repeatable, automatable, and verifiable (Humble & Farley, 2010), and that the configuration that changes between environments must reside in the environment and not in the code (Wiggins, 2017).

4.1 Meaning of the Term "Implementation"

In the context of the ADSO program, "software implementation" means **putting a release into operation** in the environment where its users will use it. In ISO/IEC/IEEE 12207:2017, that activity corresponds mainly to the **Transition** process (6.4.10), supported by the Integration (6.4.8), Verification (6.4.9), Validation (6.4.11), and Operation (6.4.12) processes, and by the technical management processes of Configuration Management (6.3.5), Risk Management (6.3.4), and Quality Assurance (6.3.8) (International Organization for Standardization [ISO] et al., 2017). The standard uses the word *implementation* (6.4.7) for the construction of the software element; to avoid confusion, this document calls that activity "building" and calls "implementation" the putting into operation. Section 8.3 relates each phase to its process.

4.2 Sources of This Document

The procedure was verified against the code of the eleven repositories of the project (*develop* branch as of October 21, 2026), especially *lavarapido-infra/docker-compose.yml*, *docker-compose.prod.yml*, *.env.example*, and the *deploy* folder, and against the deployment, environments, continuous integration, operation, quality, and project control documentation (Vehicle Washing Development Team, 2026a, 2026b). The detailed installation, administration, and use procedures are not repeated here: they are referenced in the Installation Manual, the Administrator Manual, and the User Manual, version 4.0, and in the Deployment Document of this same folder, which records how the public server was set up.

5. Objective

5.1 General Objective

To establish an orderly, verifiable, and reversible procedure for putting a release of the Vehicle Washing Management System into operation in its target environment, with the initial business data loaded, the users trained, and the formal acceptance of the business owner.

5.2 Specific Objectives

- Define the roles, the responsibilities, and the implementation strategy.
- Set the entry and exit criteria that determine when the implementation can start and when it is considered finished.
- Describe, by phases, the preparation, deployment, initial configuration, verification, acceptance, training, and go-live steps, with their person responsible and their evidence.
- Establish a rollback plan that allows returning to the previous release without data loss.
- Identify the risks of the implementation and their treatment measures.

6. Scope

6.1 What This Procedure Covers

- The implementation of a tagged release of the backend (six services and gateway), the database, the messaging, and the web application on a machine with Docker in the local network or on the public server (an Azure virtual machine with Docker, Caddy, and

HTTPS that the team turns on when it is needed). **Warning:** the development *docker-compose.yml* publishes on the machine ports 1433 (SQL Server with the *sa* account), 5672 and 15672 (RabbitMQ), 1025 and 8025 (Mailpit), and 3001 to 3006 (each service without going through the gateway); on the local network any computer can reach them. For real use the public server with *docker-compose.prod.yml* is recommended, since it publishes none of them; if the local network is used, block those ports in the machine's firewall and leave only 8080 and 4200 open.

- The initial implementation (first go-live at the car wash or on the evaluation machine) and later implementations of new releases.
- The load of the master data of the business (opening hours, bays, services, prices, points, payment accounts, operators, and promotions).
- The smoke tests, the acceptance tests, the training, the go-live, the post-implementation support, and the rollback.

6.2 What This Procedure Does Not Cover

- A high-availability production environment (several replicas, database managed by the provider): the documentation declares it outside the academic scope. Section 14 lists what would be missing to reach it. The step-by-step setup of the public server is in the Deployment Document.
- The publication of the mobile application in app stores; the Android application is distributed as an APK from the public server (button and QR code on the home page).
- Setting up continuous integration: since October 19, 2026, each repository builds and runs its tests with GitHub Actions on every change, and the server update can also be launched from GitHub (Deployment Document). Phase 2 is kept to verify the release before implementing it.
- The migration of historical data: the business has no previous system; the paper schedule is not digitized (section 8.1).

7. Roles and Responsibilities

Table 2 assigns the responsibilities with the RACI convention: R (responsible for doing the work), A (approves and is accountable), C (consulted), and I (informed).

Table 2

RACI Matrix of the Implementation

Activity	Project leader (Brayan Estiven Patiño Cabrera)	Backend leader (Merari Urbano Quintero)	Frontend leader (Juan Camilo Guilombo Osorio)	IT administrator	Business owner or instructor
Planning and approval of the window	A	C	C	C	I
Environment preparation	I	C	I	R	—
Building and verifying the release	A	R	R	I	—
Pre-implementation backup	I	C	—	R	—
Deployment of infrastructure, database, and services	A	R	I	R	—
Deployment of the clients	A	I	R	C	—
Initial business configuration	A	C	C	R	C
Smoke tests	A	R	R	R	—
Acceptance tests	R	C	C	C	A
Training	A	R	R	C	I
Go-live and post-implementation support	A	R	R	R	I
Rollback decision	A	R	R	R	I
Closure and lessons learned	R	C	C	C	I

Note. The IT administrator is not an application role: it is the person who operates the server, Docker, and SQL Server. In the academic team, this role is taken by the person designated by the project leader. Prepared by the authors.

8. Implementation Strategy

8.1 Starting Point

Before the system, the car wash did not use software: bookings were written down by hand, slots overlapped, and cash payments left no record of the customer. For that reason, **there is no legacy data to migrate** and no previous system that must coexist with the new one. The data that the system needs to operate is master data that the administrator loads from the panel (section 11.8), plus the catalogs that the migrations load automatically (roles, vehicle types, statuses, categories, notification types, and three sample services).

8.2 Selected Strategy

A **direct implementation by technical phases, with a controlled pilot period**, is adopted:

1. **Technical phases.** First the infrastructure and the database, then the services and the gateway, then the web application. Each phase has its verification, and the procedure does not advance if it fails (section 11).
2. **Pilot.** During the first week of operation, the system is used with a reduced group of customers and with all the operators, while the administrator keeps the paper schedule as a contingency backup (not as a parallel system: the official source is the system).
3. **Cutover.** Once the pilot ends without open incidents of severity P0 or P1, registration is opened to all customers and the paper schedule is retired.

A full parallel implementation (operating on paper and in the system at the same time for weeks) is discarded because it duplicates the work of the staff without providing comparability: the previous process does not produce data that can be reconciled with the system's data. An implementation by functional modules is discarded because the flows depend on each other (there is no payment without a booking and no assignment without an operator).

8.3 Relationship With the ISO/IEC/IEEE 12207 Processes

Table 3

Phases of the Procedure and Processes of ISO/IEC/IEEE 12207:2017

Phase	Main process of the standard	Supporting processes
0. Planning and approval	Transition (6.4.10): prepare for the transition	Project planning (6.3.1), Decision management (6.3.3)
1. Environment preparation	Transition (6.4.10)	Infrastructure management (6.2.2)
2. Building and verifying the release	Integration (6.4.8) and Verification (6.4.9)	Configuration management (6.3.5)
3. Pre-implementation backup	Transition (6.4.10)	Risk management (6.3.4)
4 to 6. Deployment	Transition (6.4.10): perform the transition	Configuration management (6.3.5)
7. Initial business configuration	Transition (6.4.10)	Information management (6.3.6)
8. Smoke tests	Verification (6.4.9)	Quality assurance (6.3.8)
9. Acceptance tests	Validation (6.4.11)	Quality assurance (6.3.8)
10. Training	Transition (6.4.10) and Operation (6.4.12)	Knowledge management (6.2.6)
11. Go-live and support	Operation (6.4.12)	Maintenance (6.4.13)
12. Closure	Project assessment and control (6.3.2)	Knowledge management (6.2.6)

Note. Prepared by the authors from ISO et al. (2017). The numbers in parentheses are the clauses of the standard.

9. Target Environment and Components

The target environment is a machine or server with Docker (Windows 10 or 11 with WSL 2, Ubuntu 22.04 or later, or macOS) that runs all the components of Table 4 with *docker compose --profile app*. The hardware and software requirements and the ports are those of sections 6 and 7 of the Installation Manual (minimum 4 cores, 8 GB of RAM with 5 GB for Docker, and 20 GB of free disk; recommended 16 GB of RAM and SSD). On the public server *docker-compose.prod.yml* is added: the web is served compiled from a container with nginx, Caddy publishes only ports 80 and 443 with HTTPS, and no internal service is exposed. Without that file (local network), 1433, RabbitMQ, Mailpit, and 3001 to 3006 stay published: see the warning in the scope section.

Table 4

Components That Are Implemented

Component	Source	Port	Implemented in
sqlserver	Image *mcr.microsoft.com/mssql/server:2022-latest*	1433	Phase 4
db-init	Same image; runs once and creates the *LavaRapido* database	—	Phase 4
payment-migrate	Image *liquibase/liquibase:4.29*; applies the payment changelogs	—	Phase 4
rabbitmq	Image *rabbitmq:4-management*	5672 and 15672	Phase 4
mailpit (tests only) or real SMTP	Image *axllent/mailpit* or Gmail	1025 and 8025	Phase 4
security-service	Repository *lavarapido-security-service* (Dockerfile)	3001	Phase 5
customer-service	Repository *lavarapido-customer-service*	3002	Phase 5
booking-service	Repository *lavarapido-booking-service*	3003	Phase 5
operations-service	Repository *lavarapido-operation-service* (singular name)	3004	Phase 5
payment-service (.NET 8)	Repository *lavarapido-payment-service* (Dockerfile)	3005	Phase 5
notification-service	Repository *lavarapido-notification-service*	3006	Phase 5
api-gateway	Repository *lavarapido-api-gateway*	8080	Phase 5
Web application	Repository *Front-end-proyecto-web*	4200 in the local network; container with nginx behind Caddy on the public server	Phase 6
Mobile application	Repository *LavadoVehicular-Movil*	APK built with EAS and published at */app/* on the server; 8081 (Metro) in development	Phase 6

Note. Prepared by the authors from *docker-compose.yml* and *docker-compose.prod.yml* (October 16, 2026).

10. Entry Criteria

The implementation **does not start** until all the criteria of Table 5 are met. The project leader verifies each one and records the evidence in the log (Annex D).

Table 5

Entry Criteria

No.	Criterion	Evidence
E1	The code of the release is merged into *develop* (or *main*) of the eleven repositories, with no uncommitted work.	Clean *git status* and *git log* with the last commit of each repository.
E2	The automated tests pass: *./mvnw -B verify* in the Java services and the gateway, *dotnet test* in payment-service, *ng test* in the web, *npx jest* in the mobile app, and the end-to-end suite of *lavarapido-infra/e2e* against the containers, which is run locally because the CI only checks its syntax.	Summaries without failures. Latest *develop* CI (October 24, 2026): 1,119 automated tests in the web, the gateway, and the six services (11 payment integration tests skipped without SQL Server) and 73 in the mobile app, 1,192 in total; the local end-to-end suite (19 tests) last ran on October 21, 2026, and the remote battery (*e2e.yml*, 44 cases) has not been run against the public server yet.
E3	The web application builds in production mode (*npx ng build*).	Folder *dist/* generated without errors.
E4	The release is tagged in each repository (for example, *v1.0.0*) (section 13). As of October 24, 2026, *v1.0.0* exists in 9 repositories (payment-service has it since that day, with its PR #2 merged); the mobile app has no tag yet (PR #1 open) and is implemented from *develop* until its first release (exception recorded in the corrective actions document, ACO-24). The public server runs *develop* of October 24, which will ship as *v1.1.0*.	*git tag* in each repository.
E5	The installation, administrator, and user manuals correspond to the release (version 4.0 or later).	Version control of the manuals.
E6	Open incidents of severity P0 and P1 are resolved; P2 and P3 incidents are recorded with their action in the corrective, preventive, and improvement actions document.	Updated action register. As of October 24, 2026 it is met: no P0 or P1 is open and the 16 open ones (P2 and P3) have an action (Actions document, Table 10).
E7	The target environment meets the hardware, software, and port requirements.	Checklist of Annex A.
E8	The production secrets are generated and stored outside the repository (*DB_PASSWORD*, *JWT_SECRET*, *INTERNAL_API_KEY*, *RABBITMQ_PASSWORD*, and the mail app password).	Verbal confirmation of the IT administrator; never a copy of the value.
E9	If the environment already has data, there is a verified backup less than 24 hours old (phase 3).	*.bak* file and output of RESTORE VERIFYONLY; on the public server, the *RESPALDO | OK* and *VERIFICACIÓN | OK* lines of *~/lavarapido/backups/backup.log*.
E10	The implementation window is approved and communicated (section 15).	Approval message of the project leader.

Note. Prepared by the authors from the definition of done and the testing strategy of the project (Vehicle Washing Development Team, 2026a).

11. Step-by-Step Procedure

Each phase states its person responsible, its estimated duration, the steps, and the evidence recorded in the log (Annex D). The durations are estimates of this document for a first implementation on a machine that meets the requirements; later implementations of a new release take less time because phases 1 and 7 are shorter.

11.1 Phase 0: Planning and Approval

Responsible: project leader. **Estimated duration:** 1 hour.

1. Define the release to be implemented (tag) and the scope of changes compared with the previous one.
2. Choose the window: outside the opening hours of the car wash (Monday to Saturday, 7:00 a.m. to 6:30 p.m. in the initial data), with at least three hours available.
3. Review the risks of section 14 and confirm the measures.
4. Confirm the people for each role of Table 2 and a team communication channel.
5. Communicate the window to the people affected (section 15).

Evidence: record of the release, the window, and the participants in the log.

11.2 Phase 1: Environment Preparation

Responsible: IT administrator. **Estimated duration:** 1 to 2 hours (first time).

1. Install and check Git, Docker with Compose v2, JDK 21, the .NET 8 SDK, and Node.js 22 LTS (Installation Manual, section 7.1). On Windows, confirm that Docker uses the Linux engine with WSL 2.
2. Check that ports 1433, 5672, 15672, 1025, 8025, 3001 to 3006, 8080, and 4200 are free. If a native Windows SQL Server uses port 1433, stop the *MSSQLSERVER* service.
3. Clone the repositories **side by side and with their original names**, at the release tag. As of October 24, 2026, tag *v1.0.0* exists in 9 repositories: payment-service has it since that day (its PR #2 was merged and its *main* includes the whole *develop*, so cloning it from *develop*, as in the block below, gives the same result); only the mobile app has no release. The fixes of October 23 and 24 are in *develop*, which is what runs on the public server since October 24, and will reach *main* with *v1.1.0*: to install that version earlier, replace *-b v1.0.0* with *-b develop* (on the public server, *BRANCH=develop*; without it, *setup-server.sh* stops because it never downgrades). The operations repository is called *lavarapido-operation-service* (singular), and *docker-compose.yml* builds it with that name: cloning it with another name makes the build fail.

```
mkdir Backend && cd Backend
for repo in lavarapido-infra lavarapido-security-service lavarapido-api-gateway \
            lavarapido-notification-service lavarapido-operation-service; do
  git clone -b v1.0.0 https://github.com/BrayanCABRERA123/$repo.git
done
git clone -b v1.0.0 https://github.com/camiloguilombo12/lavarapido-customer-service.git
git clone -b v1.0.0 https://github.com/camiloguilombo12/lavarapido-booking-service.git
git clone -b develop https://github.com/merari0322/lavarapido-payment-service.git   # no tag yet
cd .. && mkdir Frontend && cd Frontend
git clone -b v1.0.0 https://github.com/BrayanCABRERA123/Front-end-proyecto-web.git
```

1. Create the configuration file and check that Git ignores it:

```
cd ../Backend/lavarapido-infra
cp .env.example .env
git check-ignore -v .env      # must show the .gitignore rule
```

1. Fill in the *.env* file with the implementation values of Table 6.

Table 6

Configuration Values for the Implementation

Variable	Implementation value	Reason
DB_PASSWORD	New strong password (8 or more characters with uppercase, lowercase, number, and symbol).	SQL Server rejects weak ones; ne reuse the development password.
JWT_SECRET	Random value of at least 32 bytes (*openssl rand -base64 48*).	Signs the tokens; the same for all
INTERNAL_API_KEY	Random value (*node -e "console.log(require('crypto').randomBytes(32).toString('base64url'))"*).	Without it, booking and payment and reminders are not sent by ema is no automatic assignment and, si *develop* of October 23, 2026 (E-0 operator cannot start or finish serv Since October 24 customer-servic receives it (together with *BOOKING_SERVICE_URL*, fixed compose file) to ask booking-serv whether a vehicle has active book production the compose file does without it. Since October 26, 2026 payment-service also requires it (*compose.prod.yml*): without it, bo service cannot check with paymer whether an overdue digital bookin already has a receipt, and it cancel booking for lack of payment.

Variable	Implementation value	Reason
SPRING_PROFILES_ACTIVE	*prod* (any value other than *dev*).	With *dev* (the default value of the Compose file), Swagger is publish the demonstration accounts whose password is in *DEMO_ADMIN_PASSWORD*, *DEMO_OPERATOR_PASSWORD* *DEMO_CLIENT_PASSWORD* are (empty: that account is not created
ALLOWED_EMAIL_DOMAINS	*gmail.com* (default) or the business's list of domains, comma separated.	Since *develop* of October 23, 2026 01), security-service only accepts domains when registering, creating and changing the email (400 *EMAIL_DOMAIN_NOT_ALLOW* since October 24 sign-in accepts a domain.
AUTH_RATE_LIMIT_*, TRUSTED_PROXIES, ROUTE_CONNECT_TIMEOUT, ROUTE_READ_TIMEOUT	Optional (api-gateway).	Since October 24, 2026: 60 reques minute per IP on */api/v1/auth/** (*AUTH_RATE_LIMIT_ENABLED* *_REQUESTS*, and *_WINDOW*: *tru* and *1m*), the client's real IP only fi private networks (*TRUSTED_PRO* and 5 s to connect and 60 s to read each route. They have a default va the service and are not yet in the c file or *.env.example*: to change the them to the gateway's *environmen*
START_LATE_LIMIT	Optional (operations-service).	Since October 24, 2026, how long the end time the operator can start service (default *PT4H*; afterwards *SERVICE_TOO_LATE*). Like the ones, it is not in the compose file.
MESSAGING_ENABLED	*true*	With *false* (the default value), no notifications, point notices, or eve driven profiles are generated.
RABBITMQ_USERNAME, RABBITMQ_PASSWORD	New user and password (not *guest*).	The RabbitMQ console must not h default credentials.
MAIL_ENABLED, MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_FROM, MAIL_DOCKER_HOST	*true*, *smtp.gmail.com*, *587*, business account, app password, and *smtp.gmail.com* for Docker.	Mailpit only catches emails; real operation uses a real SMTP server
CORS_ALLOWED_ORIGINS	Real origin of the web application (for example, *https://<domain>* on the public server or *http://lavadero-server:4200* in the local network).	Browsers reject other origins.
ADMIN_EMAIL, ADMIN_PASSWORD	Email and password of the first administrator (only the first time).	security-service creates the accour startup if there is still no administr
PUSH_ENABLED	*true* when the installed mobile application is the APK built with EAS.	Expo Go does not receive remote notifications.
BOOKING_ZONE	*America/Bogota*	Time zone of the car wash; the sys stores UTC.

Variable	Implementation value	Reason
SESSION_CHECK_*, BOOKING_EXPIRY_*, FOLLOW_UP_*	Optional (commented out in *.env.example* and *.env.production.example*).	They have defaults (gateway sessi check on with 30 s and 3 s; a 60-n grace after the end, *BOOKING_EXPIRY_GRACE_MI BOOKING_EXPIRY_CHECK_EN* and *BOOKING_EXPIRY_CHECK_IN* have no effect since October 24, 2 because booking-service stopped cancelling bookings on its own; operations-service follow-up ever minutes for bookings that ended a nobody closed in the last 28 days, *FOLLOW_UP_LOOK_BACK_DA* which uses the same grace and *INTERNAL_API_KEY*). There are and *docker-compose.yml* passes th the containers.
BOOKING_CASH_ACTIVE_LIMIT, BOOKING_PAYMENT_WINDOW, BOOKING_PAYMENT_EXPIRY_ENABLED, BOOKING_PAYMENT_EXPIRY_INTERVAL	Optional (booking-service; commented out in *.env.example*).	Since October 26, 2026 (payment booking): at most 1 active cash bo per customer, 30 minutes to uploa receipt of a booking with digital p and the task that cancels overdue c and every 60 seconds (*1*, *30m*, *tru 60s*). *docker-compose.yml* passes t the container, together with *PAYMENT_SERVICE_URL* (fixed *http://payment-service:3005*).

Note. Prepared by the authors from *.env.example* and *docker-compose.yml*. Secret values are written only in the *.env* file; never in a message, a shared document, or a repository.

Evidence: output of the version commands, list of repositories cloned at the tag, and confirmation that *.env* is ignored.

11.3 Phase 2: Building and Verifying the Release

Responsible: backend leader and frontend leader. **Estimated duration:** 30 to 45 minutes.

1. Run the tests of each Java service and of the gateway (*./mvnw -B verify*; on Windows, *.\mvnw.cmd -B verify*).
2. Run the tests of payment-service (*dotnet test* from *lavarapido-payment-service*).
3. Run the web tests (*npx ng test --watch=false*) and build it (*npm ci* and *npx ng build*).
4. Validate the Compose file and build the images:

```
cd Backend/lavarapido-infra
docker compose config --quiet && echo "Compose file is valid"
docker compose --profile app build
```

Evidence: test summaries (number of tests and zero failures), output of *ng build*, and the list of *lavarapido/*:local* images built.

11.4 Phase 3: Pre-Implementation Backup

Responsible: IT administrator. **Estimated duration:** 15 minutes. **Applies** when the environment already has data (later implementations).

On the public server this phase is automatic: *deploy/update-server.sh* first runs *deploy/backup-db.sh* (full backup with a checksum, *RESTORE VERIFYONLY*, and a compressed copy in *~/lavarapido/backups/*, where the last 14 are kept) and, if the backup fails, does not update (Deployment Document, section 3.2). The following steps are for a computer on the local network.

1. Generate a full backup with a checksum and verify it (Administrator Manual, section 14.3):

```
docker compose exec -e SQLCMDPASSWORD="$DB_PASSWORD" sqlserver /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -C -b -Q \
  "BACKUP DATABASE [LavaRapido] TO DISK = N'/var/opt/mssql/data/pre_v1.1.0.bak' WITH INIT, CHECKSUM"
docker compose exec -e SQLCMDPASSWORD="$DB_PASSWORD" sqlserver /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -C -b -Q \
  "RESTORE VERIFYONLY FROM DISK = N'/var/opt/mssql/data/pre_v1.1.0.bak' WITH CHECKSUM"
docker cp lavarapido-sqlserver:/var/opt/mssql/data/pre_v1.1.0.bak ./backups/
```

The command does not use *COMPRESSION*: SQL Server Express, the edition on the public server, does not support it and the backup would fail; *backup-db.sh* compresses afterwards with *gzip*. The password is given to *sqlcmd* in the *SQLCMDPASSWORD* variable and not with *-P*, as *backup-db.sh* does. On the public server this command is not written: use *bash ~/lavarapido/lavarapido-infra/deploy/backup-db.sh* and, to prove that the backup works, *deploy/verify-backup.sh*.

1. Keep an encrypted copy of the current *.env* file, separate from the data backup.
2. Write down the tag of the release in operation: it is the release to return to if a rollback is needed (section 12).

Evidence: name, size, and verification result of the backup (on the public server, the *RESPALDO | OK* line of *backup.log*).

11.5 Phase 4: Infrastructure and Database

Responsible: IT administrator with the backend leader. **Estimated duration:** 10 to 15 minutes.

1. Start the infrastructure:

```
cd Backend/lavarapido-infra
docker compose up -d
docker compose ps
```

1. Wait until *lavarapido-sqlserver* and *lavarapido-rabbitmq* are *healthy* and *db-init* and *payment-migrate* end with exit code 0.
2. Check that the *payment* and *promotion* schemas exist (created by *payment-migrate*). The other schemas are created by each Java service with Liquibase when it starts in phase 5.

Criterion to continue: healthy containers and payment migrations with exit code 0. If *payment-migrate* fails, do not continue (section 12).

Evidence: output of *docker compose ps*.

11.6 Phase 5: Backend Services and Gateway

Responsible: backend leader. **Estimated duration:** 10 to 20 minutes.

1. Start the services and the gateway:

```
docker compose --profile app up -d
docker compose ps
```

1. Follow the log of each Java service until Liquibase has applied its changelogs and the application has started (*docker compose logs -f security-service*, and so on with the others).
2. Check the health of the seven components:

```
for port in 3001 3002 3003 3004 3005 3006 8080; do
  echo "$port: $(curl -s http://localhost:$port/actuator/health)"
done
```

1. Check that the database has 57 tables in the nine schemas (Installation Manual, section 11.5) and that the RabbitMQ topology exists (exchange *carwash.events* and the queues *customer-service.events* and *notification-service.events*, each with its *.dlq*, and *operations-service.events*, which has no dead-letter queue).

Criterion to continue: the seven responses are *UP* and the number of tables is the expected one. Every service, including payment-service, checks the database in its health endpoint and answers *DOWN* (503) if it is not available.

Evidence: output of the health loop and of the table count.

11.7 Phase 6: Client Applications

Responsible: frontend leader. **Estimated duration:** 15 minutes.

1. **Web application.** In the local network, it is published with the Angular server, which forwards */api* to the gateway through *proxy.conf.json*:

```
cd Frontend/Front-end-proyecto-web
npm ci
npx ng serve --host 0.0.0.0    # reachable from other machines on the local network
```

On the public server the web does not use *ng serve*: the *web* container builds the *dist/* folder and serves it with nginx (with security headers) behind Caddy and with HTTPS; *deploy/update-server.sh* rebuilds it after each push (Deployment Document).

1. **Mobile application.** The APK is built with EAS with *EXPO_PUBLIC_API_URL* pointing to the server domain and is published with *npm run publish:apk*, which leaves it at */app/LavaRapidoVehicular.apk* and notifies the people who have the application by push. For tests in the local network, *npx expo start* can be used with the IPv4 address of the machine. The version of October 17, 2026, adds the native modules *expo-image-picker* and *expo-image-manipulator* (inspection report photos), so it requires building and publishing a new APK; an over-the-air update (*npm run publish:update*) is not enough, because it only changes the screen code. The app changes of October 21, 2026 (compressed receipt, QR magnifier, and mandatory update screen) add no native modules and do arrive with *npm run publish:update*; since then, if the server publishes an APK with a higher *versionCode*, the app asks to install it before going on.

Evidence: screenshot of the welcome screen served from the server, with the address bar visible.

11.8 Phase 7: Initial Business Configuration

Responsible: IT administrator with the business administrator. **Estimated duration:** 1 to 2 hours.

Because the *dev* profile is disabled, there are no demonstration accounts. The master data is loaded in the order of Table 7, because each step depends on the previous ones.

Table 7

Load Order of the Master Data

Step	Data	Where it is loaded	Reference
1	First administrator account	Variables *ADMIN_EMAIL* and *ADMIN_PASSWORD* of the environment file; security-service creates it at startup	Administrator Manual, section 8.4
2	Weekly hours, break, and holidays of the year	Hours & Bays > Business hours	Administrator Manual, section 11.2
3	Washing bays and their status	Hours & Bays > Wash bays	Administrator Manual, section 11.3
4	Services, prices, and duration by vehicle type, and loyalty points	Management > Services	Administrator Manual, section 12.2
5	Payment accounts with their QR code	Settings > Payment methods	Administrator Manual, section 11.4
6	Operator accounts and other administrators	Management > Users or Operators > New operator	Administrator Manual, section 9.3
7	Weekly shift of each operator	Operators > Edit availability	Administrator Manual, section 12.1
8	Promotions (discount coupons unlocked with points)	Management > Promotions	Administrator Manual, section 12.8
9	Verification of the payment accounts	Scan each QR code from a test customer account and confirm the holder	Administrator Manual, section 11.4
10	Business data (legal name, tax ID, address, phones, and channels)	Settings > Business data	Administrator Manual, section 11.1

Note. The business data are saved in booking-service and are the data customers see on the home page and in the help center. Prepared by the authors.

Evidence: screenshot of each configured screen and list of the accounts created (without passwords).

11.9 Phase 8: Smoke Tests

Responsible: backend and frontend leaders and IT administrator. **Estimated duration:** 20 minutes.

The smoke tests confirm that the release works end to end before handing it over for acceptance. Table 8 is run with test accounts that are deactivated at the end.

Table 8

Smoke Tests

No.	Test	Expected result
H1	Protected route without a token through the gateway (*curl -i http://localhost:8080/api/v1/bookings/me*).	401 returned by booking-service.
H2	Registration of a test customer and login.	Panel */client*; welcome email received.
H3	Password recovery.	Six-digit code received by email.
H4	Registration of a vehicle and a booking at a free time.	Booking confirmed with a bay and the points shown in the summary.
H5	Query of the payment accounts from the payment screen.	The active accounts are listed with their QR code (confirms the connection of payment-service to the database).
H6	Booking without choosing an operator and booking choosing a free one.	The system assigns an operator in both cases (the chosen one in the second); notice to the operator.
H7	Start and finish of the service by the operator.	Booking COMPLETED; notifications to the customer.
H8	Payment report with a test receipt and approval by the administrator.	Payment approved; payment confirmed email and points credited.
H9	Dead-letter queues.	*notification-service.events.dlq* and *customer-service.events.dlq* at zero.
H10	Sign out and repeat a call with the same token (*curl -i -H "Authorization: Bearer \<token\>" https://\<domain\>/api/v1/bookings/me*).	401: the gateway cuts the token of the closed session (version of October 21, 2026 or later).
H11	With the administrator's token, query the bookings to close (*GET /api/v1/admin/assignments/to-close*) and open the administrator's home.	200 with the list (it may be empty); the "To close" card only appears when something is pending (version of the evening of October 21, 2026 or later).

Note. Prepared by the authors from the checklist before a review session of the project (Vehicle Washing Development Team, 2026a).

Criterion to continue: the eleven tests pass. A failure in H1 to H8, H10, or H11 triggers the rollback evaluation (section 12).

11.10 Phase 9: Acceptance Testing

Responsible: project leader. **Approves:** business owner or instructor. **Estimated duration:** 1 to 2 hours.

1. Run, with the business owner, the acceptance cases of Annex B, which cover the flows of the three roles with the real master data.
2. Record the result of each case (passed or failed) and the remarks.
3. Classify each remark with the severity of the project's incident management (P0 to P3) and record it in the corrective, preventive, and improvement actions document.
4. Sign the acceptance record (Annex C). Acceptance can be **full** (all cases passed), **conditional** (only P2 or P3 remarks, with a correction date), or **rejected** (a case failed because of a P0 or P1 defect).

Evidence: signed acceptance record and results of the cases.

11.11 Phase 10: Training

Responsible: backend and frontend leaders. **Estimated duration:** 3 hours in total.

Table 9

Training Plan by Role

Audience	Content	Material	Duration
Business administrator	Panel, bookings, manual operator assignment, review, approval, and refund of payments, services, prices, and points, promotions, hours, bays, reports, and notifications.	Administrator Manual, sections 8 to 13	90 minutes
IT administrator	Start and stop, logs, backup and restoration, preventive maintenance, incidents, and rollback.	Administrator Manual, sections 14 to 17; this procedure, section 12	45 minutes
Operators	Schedule, assigned services, start and finish, notifications, and ratings.	User Manual, section 10.2	30 minutes
Customer service staff	Registration, booking, payment by QR code, coupons and points, history, and rating, to guide customers.	User Manual, section 10.1	30 minutes

Note. Each session ends with a practical exercise in the implemented environment with test accounts. Prepared by the authors.

Evidence: attendance list with date, topic, and signature.

11.12 Phase 11: Go-Live and Post-Implementation Support

Responsible: the whole team. **Duration:** one-week pilot and two weeks of reinforced support.

1. Deactivate the test accounts created in phases 8 and 9.
2. Enable the system for the pilot group and then for all customers (section 8.2).
3. During the first two weeks, the IT administrator performs the tasks of Table 10 every day, and the team handles incidents with the times of the project's incident management (P0 immediately; P1 the same day).

Table 10

Daily Monitoring During Post-Implementation Support

Task	How it is checked
Health of the seven components	Loop of section 11.6.
Errors of the last 24 hours	*docker compose logs --since 24h | grep -i error*
Dead-letter queues	RabbitMQ console (*http://localhost:15672*).
Payments to review and bookings without an operator	Administrator panel.
Daily backup generated and verified	File of the day and backup log; on the public server, *tail -5 ~/lavarapido/backups/backup.log* with the day's *RESPALDO | OK* and *VERIFICACIÓN | OK*, and *grep FALLÓ* with no results.
Container status (public server)	*docker compose ps*: all *healthy*.
Disk space	*docker system df*; usage below 80%.

Note. Adapted from the preventive maintenance plan of the Administrator Manual, section 17.6.

Evidence: daily record of the monitoring and of the incidents handled.

11.13 Phase 12: Closure

Responsible: project leader. **Estimated duration:** 1 hour.

1. Confirm the exit criteria (section 16).
2. Hold a retrospective of the implementation and record the lessons learned in the corrective, preventive, and improvement actions document.
3. Update the risk register, the technical backlog, and the manuals if the implementation revealed differences.
4. File the log, the acceptance record, and the attendance lists with the implemented release.

12. Rollback Plan

The rollback returns the environment to the previous release when the new one cannot operate. It is decided with the criteria of Table 11 and performed by the backend leader with the IT administrator.

Table 11

Rollback Criteria

Situation	Decision
payment-migrate or a Liquibase migration of a Java service fails	Roll back immediately.
A smoke test H1 to H8, H10, or H11 fails and is not fixed within 30 minutes	Roll back.
P0 incident in the first 24 hours of operation	Roll back.
P1 incident without a fix or a workaround on the same day	Evaluate with the project leader; roll back if it affects bookings or payments.
P2 or P3 remarks	Do not roll back; fix in a later release.

Note. Prepared by the authors from the severity classification of the project.

Rollback steps:

1. Communicate the decision and stop user access.
2. Stop the services and the gateway, leaving SQL Server running:

```
docker compose stop api-gateway security-service customer-service booking-service \
  operations-service payment-service notification-service
```

1. Restore the pre-implementation backup of phase 3 (Administrator Manual, section 15.2). On the public server, *bash deploy/restore-db.sh \<backup.bak.gz\>* is used: it asks to type *RESTAURAR*, first backs up the current database, stops the services, restores, and starts them again. Liquibase changesets only move forward, so **a migration is not undone with another one**: the database is restored to its state before the release.
2. In each affected repository, return to the previous version written down in phase 3: its tag (the first one that exists is *v1.0.0*; there are no earlier tagged versions) or, if it has none, its commit (*git checkout \<tag or commit\>*); then rebuild and start: *docker compose --profile app up -d --build*. On the public server it is not done this way, because *update-server.sh* pulls the tip of the branch again: revert the change on GitHub (*git revert* in a branch with its PR) and deploy again or, in an emergency, go back to the images *update-server.sh* kept as *:prev* with *bash deploy/rollback.sh* (all services) or *rollback.sh \<service\>*, which asks to type *VOLVER*, recreates the containers without building, and waits for them to be healthy (Deployment Document, section 3.4). Since October 24, 2026 *restore-db.sh* runs *payment-migrate* at the end and *setup-server.sh* never downgrades without *ALLOW_DOWNGRADE=1*.
3. Restore the previous *.env* file if the release had added variables.
4. Repeat the smoke tests of section 11.9.
5. Communicate the restoration, stating the date and time of the last recovered data; payments received after the backup are reconciled with the bank statement.
6. Record the rollback as a nonconformity in the corrective, preventive, and improvement actions document, with its root cause analysis.

Target rollback time: 1 hour from the decision. **Maximum data loss:** the records after the backup of phase 3.

13. Release Configuration Management

- **Tagging.** Each implemented release is tagged in the code repositories with the same name (semantic versioning: *vMAJOR.MINOR.PATCH*, for example, *v1.0.0*). Tags make it possible to rebuild exactly what was implemented and to roll back. Status as of October 24, 2026: *v1.0.0* in 9 repositories (web, api-gateway, security, customer, booking, operations, notification, and infra since October 23, and payment-service since the 24th, with its PR #2 merged); the mobile app is missing (PR #1 open), and *develop* already has the fixes of October 23 and 24 for *v1.1.0*.
- **Branches.** Releases are implemented from *main*, which has the tagged version; payment-service, while it has no release, from *develop*; never from a *feature/** branch. The public server runs *develop* of October 24, 2026 and is updated with *BRANCH=develop* until *v1.1.0*; afterwards, continuous deployment deploys *main* (Deployment Document, section 4.3.4).

- **Images.** The Compose file builds *lavarapido/\<service\>:local* images. To keep the previous release, each image is tagged before rebuilding (*docker tag lavarapido/booking-service:local lavarapido/booking-service:prev*, or with the name of the current version, *:v1.0.0*); to go back, do the reverse and *docker compose up -d --no-build \<service\>*. On the public server they do it on their own since October 24, 2026: *update-server.sh* tags *:prev* before rebuilding and *rollback.sh* goes back to that image.
- **Database.** The schema is versioned with Liquibase; each service keeps its history in *DATABASECHANGELOG_\<SERVICE\>* (payments in *DATABASECHANGELOG_PAYMENT*). An executed changeset is never edited.
- **Configuration.** *.env.example* is versioned; *.env* is not. Every new variable is added to *.env.example* with a comment and to the *environment* block of the service in *docker-compose.yml*. Those of October 21, 2026 (*SESSION_CHECK_**, *BOOKING_EXPIRY_**, and *FOLLOW_UP_**, nine variables) already follow this rule (action ACO-14, closed on October 23, 2026), as do *ALLOWED_EMAIL_DOMAINS* and *DEMO_*_PASSWORD* of October 23. Those of October 24 in the gateway (*AUTH_RATE_LIMIT_**, *TRUSTED_PROXIES*, *ROUTE_*_TIMEOUT*) and in operations-service (*START_LATE_LIMIT*) have a default value but are not yet in *.env.example* or the compose file (ACO-40).
- **Documentation.** The version control of each manual records the software release it corresponds to.

14. Implementation Risks

Table 12

Implementation-Specific Risks and Their Treatment

ID	Risk	Probability	Impact	Treatment
R-I01	The operations repository is cloned as *lavarapido-operations-service* and the Compose build fails.	Medium	High	Clone with the original name *lavarapido-operation-service* (phase 1).
R-I02	*MESSAGING_ENABLED=false* or an empty *INTERNAL_API_KEY* remain, and notifications and emails do not arrive.	Medium	Medium	Values of Table 6 and tests H2, H7, and H8.
R-I03	The release is implemented with the *dev* profile, leaving demonstration accounts and Swagger exposed.	Medium	High	*SPRING_PROFILES_ACTIVE=prod*; Annex A.
R-I04	The public server is off when it is needed (it is turned on on demand).	Medium	Medium	Tell the DevOps owner in advance and run *deploy/update-server.sh* after the last push. With continuous deployment (active since October 26, 2026, AC-90), what enters *main* while the machine is off is deployed on its own when it is turned on (*deploy/boot-update.sh*, installed on the server that day); for the presentation the machine is turned on beforehand and *main* is frozen.
R-I05	There is no request rate limit in the gateway or in Caddy (TD-022); account lockout only protects each account separately.	Medium	Medium	Long and unique passwords for the administrators; review the Caddy logs.
R-I06	A wrongly loaded QR code or payment account sends the money to a third party.	Low	High	Step 9 of Table 7.
R-I07	Data loss caused by *docker compose down -v* or a damaged volume.	Low	High	Pre-implementation backup (phase 3) and daily backup, both verified; on the public server they are automatic (Deployment Document, section 3.2).
R-I08	In the local network the web is served with *ng serve* and without HTTPS.	Medium	Medium	Use it only inside the car wash network; outside it, use the public server with HTTPS.
R-I09	Continuous integration does not run the end-to-end tests, which need the complete system, so a regression across services can go unnoticed.	Low	Medium	Confirm that GitHub Actions is green in the 10 repositories and run the end-to-end suite and the smoke tests before each deployment.
R-I10	The role permissions editable in the panel are mistaken for real access control (ADR-015).	Low	Medium	Explain it in the training; real authorization comes from the role in the token, and since October 21, 2026 the roles tab says so.
R-I11	If security-service does not answer, the gateway lets requests through without confirming the session, and the token of a closed session works until it expires (1 hour at most); with security-service up, up to 30 seconds.	Low	Medium	Check the health of security-service (phase 5 and Table 10) and the gateway log (*No se pudo confirmar la sesión*).

Note. Prepared by the authors from the risk register, the technical backlog, and the verification of the code (Vehicle Washing Development Team, 2026a).

To take the system to a real production environment, in addition to the above, the following would be missing: SQL Server and RabbitMQ hosted with the secrets in a provider vault, one database login per service instead of *sa*, a request rate limit, and an Outbox for the events if losing an event during a broker outage is not accepted (Vehicle Washing Development Team, 2026a).

15. Communication Plan

Table 13

Communication Plan of the Implementation

Moment	Message	Recipients	Channel	Responsible
48 hours before	Release, window, impact, and contact.	Team, business owner, and operators	Team chat and notice at the car wash	Project leader
Start of the window	Start of the implementation.	Team	Team chat	Project leader
End of each phase	Result and evidence.	Team	Team chat and log	Person responsible for the phase
Rollback decision	Reason and estimated time.	Team and business owner	Team chat	Project leader
Go-live	System available, access address, and support channel.	Car wash staff and pilot customers	Direct message and notice at the car wash (the web and the app have no screen to send general announcements; improvement action AM-13)	Business administrator
Closure	Result, lessons learned, and pending items.	Team, business owner, and instructor	Closing meeting	Project leader

Note. Prepared by the authors from the incident management of the project.

16. Exit Criteria

The implementation is considered **finished** when all the following criteria are met:

- The smoke tests of Table 8 pass in the implemented environment.
- The acceptance record is signed with full or conditional acceptance (Annex C).
- The master data of Table 7 is loaded and verified.
- The people of each role received the training (Table 9).
- There is a verified backup taken after the initial configuration, and the daily backup is scheduled.
- The post-implementation support period ended without open P0 or P1 incidents.
- The log, the record, and the lessons learned are filed with the release.

17. Glossary

Table 14

Glossary of Implementation Terms

Term	Definition
Acceptance record	Signed document in which the business owner accepts, conditions, or rejects the implemented release.
Acceptance test	Test performed by the business owner to confirm that the system meets their needs (validation).
Compose profile	Group of services that starts only when requested (*--profile app*).
Entry criterion	Condition that must be met before starting the implementation.
Exit criterion	Condition that must be met to consider the implementation finished.
Implementation log	Chronological record of the steps performed, the people responsible, and their evidence.
Implementation window	Agreed period, outside opening hours, in which the implementation is performed.
Master data	Business configuration data that the system needs to operate (hours, bays, services, prices, and payment accounts).
Pilot	Initial period of operation with a reduced group of users.
RACI	Matrix that shows who is responsible, who approves, who is consulted, and who is informed for each activity.
Rollback	Controlled return to the previous release of the software and the data.
Smoke test	Quick test that confirms that the main functions work after a deployment (verification).
Tag	Fixed name that Git assigns to a commit to identify a release.
Transition	ISO/IEC/IEEE 12207 process that establishes the capability of a system to operate in its target environment.

Note. Prepared by the authors from ISO et al. (2017) and the project documentation.

References

Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2017). *Systems and software engineering — Software life cycle processes* (ISO/IEC/IEEE 12207:2017). https://www.iso.org/standard/63712.html

Servicio Nacional de Aprendizaje. (2026). *Guía para elaborar manuales de software: Manual de usuario, manual técnico, manual de instalación y manual de administrador* [Guide for writing software manuals: User manual, technical manual, installation manual, and administrator manual] [Unpublished instructional material]. Technologist in Software Analysis and Development program.

Vehicle Washing Development Team. (2026a). *Vehicle washing management system: Project documentation* (vehicle-w-docs) [Unpublished project documentation]. GitHub. https://github.com/code-sena/vehicle-w-docs

Vehicle Washing Development Team. (2026b). *lavarapido-infra* (develop branch) [Computer software]. GitHub. https://github.com/BrayanCABRERA123/lavarapido-infra

Wiggins, A. (2017). *The twelve-factor app: III. Config*. https://12factor.net/config

18. Annexes

Annex A. Pre-Implementation Checklist

Table 15

Pre-Implementation Checklist

Control	Met
Entry criteria E1 to E10 met (Table 5)	Yes / No
Repositories cloned at the tag, side by side and with their original names (including *lavarapido-operation-service*)	Yes / No
.env created, ignored by Git, and with the values of Table 6	Yes / No
SPRING_PROFILES_ACTIVE other than *dev*	Yes / No
MESSAGING_ENABLED=true and *INTERNAL_API_KEY* defined	Yes / No
RabbitMQ credentials other than *guest*	Yes / No
Real SMTP configured and tested	Yes / No
Free ports and native SQL Server stopped	Yes / No
Verified pre-implementation backup (if the environment had data)	Yes / No
Only the gateway and the web application reachable from outside the server	Yes / No

Note. Prepared by the authors.

Annex B. Acceptance Test Cases

Table 16

Acceptance Test Cases

ID	Role	Case	Expected result	Result
CA-01	Customer	Register with valid data and accept the legal documents.	Account created and welcome email.	Passed / Failed
CA-02	Customer	Try to register with an email that does not end in @gmail.com.	Validation message; the account is not created.	Passed / Failed
CA-03	Customer	Register a vehicle with a valid plate and another with an invalid plate.	The first is saved; the second is rejected with its message.	Passed / Failed
CA-04	Customer	Book a service at a free time.	Booking confirmed with a bay, total, and the points it adds.	Passed / Failed
CA-05	Customer	Try to book a time that another person has just taken.	"Time taken" message and up to five alternatives.	Passed / Failed
CA-06	Customer	Cancel a booking before its time.	Booking cancelled and email notice.	Passed / Failed
CA-07	Administrator	Assign an available operator and see the blocking reason of an unavailable one.	Assignment confirmed; reason visible for the unavailable one.	Passed / Failed
CA-08	Operator	Start and finish the assigned service on the day of the booking.	Statuses In progress and Completed; notices to the customer.	Passed / Failed
CA-09	Customer	Pay with a QR code, attaching the receipt and the reference.	Payment in review.	Passed / Failed
CA-10	Administrator	Approve the payment after confirming the money in the bank.	Payment approved; email to the customer; points credited.	Passed / Failed
CA-11	Administrator	Reject a payment with a reason.	Payment rejected; email to the customer with the reason.	Passed / Failed
CA-12	Customer	Redeem an unlocked coupon when paying.	Discount applied to the total and coupon redeemed email.	Passed / Failed
CA-13	Customer	Rate a completed service.	Rating visible to the operator; rating twice is not allowed.	Passed / Failed
CA-14	Administrator	Register a holiday and check that the day offers no times.	The customer sees "The car wash is closed that day".	Passed / Failed
CA-15	Administrator	Export the report of the month to PDF and Excel.	Files downloaded with the data of the period.	Passed / Failed
CA-16	All	Change the language and the theme, log out, and log in again.	Preferences kept in the account.	Passed / Failed

Note. Prepared by the authors from the functional requirements and the user and administrator manuals.

Annex C. Acceptance Record Template

Table 17

Implementation Acceptance Record

Field	Content
System and release	Vehicle Washing Management System, release ____
Date and place	
Implemented environment	Server ____, access address ____
Acceptance cases run and passed	____ of 16
Remarks and severity	
Decision	Full acceptance / Conditional acceptance / Rejection
Commitments and dates (if conditional)	
Signature of the business owner or instructor	
Signature of the project leader	

Annex D. Implementation Log Template

Table 18

Implementation Log

Date and time	Phase and step	Responsible	Result (correct / with issues)	Evidence	Remarks

Annex E. Command Summary

```
# Phase 2: building and verification
./mvnw -B verify                                  # each Java service and the gateway
dotnet test                                       # lavarapido-payment-service
npx ng test --watch=false                         # Front-end-proyecto-web
node --test "e2e/*.test.js"                       # from lavarapido-infra, with the containers up
npm ci && npx ng build                            # Front-end-proyecto-web
docker compose config --quiet && docker compose --profile app build

# Phase 3: pre-implementation backup (see section 11.4)
bash ~/lavarapido/lavarapido-infra/deploy/backup-db.sh   # public server (update-server.sh does it by itself)

# Phase 4: infrastructure and database
docker compose up -d && docker compose ps

# Phase 5: services and gateway
docker compose --profile app up -d
for port in 3001 3002 3003 3004 3005 3006 8080; do curl -s http://localhost:$port/actuator/health; echo; done

# Phase 6: web application
npx ng serve --host 0.0.0.0

# Rollback (see section 12)
bash ~/lavarapido/lavarapido-infra/deploy/restore-db.sh <backup.bak.gz>   # public server
docker compose stop api-gateway security-service customer-service booking-service \
  operations-service payment-service notification-service
```